Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Transport Layer (TCP/UDP)

**Jose L. Muñoz, Oscar Esparza, Juanjo Alins, Jorge Mata**
*Telematics Engineering*
Universitat Politecnica de Catalunya (UPC)

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP

3 TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro
- Transport Layer Motivation
- Client/Server Model
- Socket API
- netstat
- netcat

2 UDP

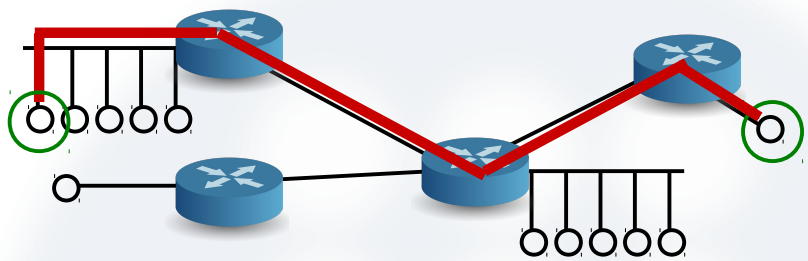
3 TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# The Network Layer

- The network layer is a scalable way of interconnecting data link layer technologies.
- Basic IP provides an interface to interface (NIC-to-NIC) best effort service for delivering datagrams.

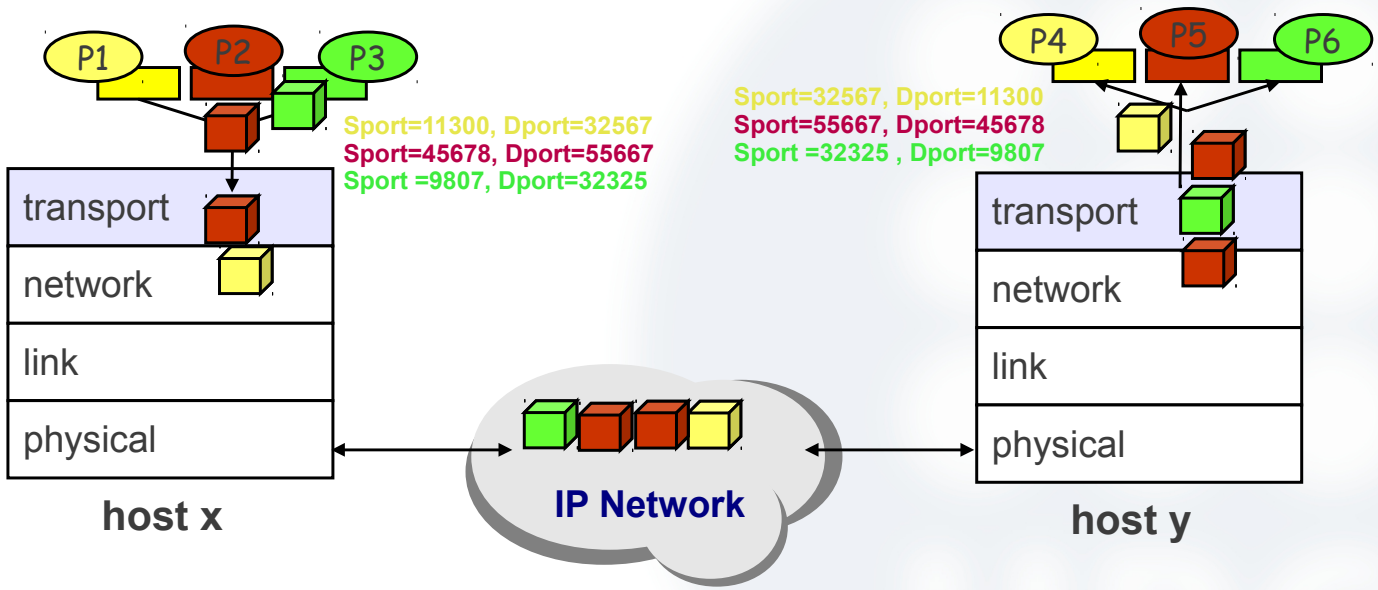


  - A best effort service means that:
    - A correct delivery of datagrams is not guaranteed.
    - There might be lost datagrams, incorrect datagrams or disordered datagrams.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Transport Layer

- The main goal: **implement communications between processes (running applications) that are in general in different systems.**
- Also called end-to-end communications.
- Introduces the concept of **PORT** for multiplexing and demultiplexing.




- The transport layer **provides the main network API** (Application Programming Interface) to allow using the network to user space processes.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Transport Ports

- Identifiying processes:
  - Each OS technology identifies its currenty running processes.
  - Unix-like systems use the Process Identifier (PID).
  - However, we want a generic identifier (different of the PID) for multiplexing transport communications.
- The **port** is a parameter for multiplexing that is dynamically assigned to any running process that requires a transport communication with another process.
- Each transport PDU carries:
  - A source Port (SPort) that identifies the process sending the PDU.
  - A destination port (DPort) that identifies the process in the destination host.
- For the main transport protocols of Internet (TCP and UDP), ports have 16 bits (65536 ports).

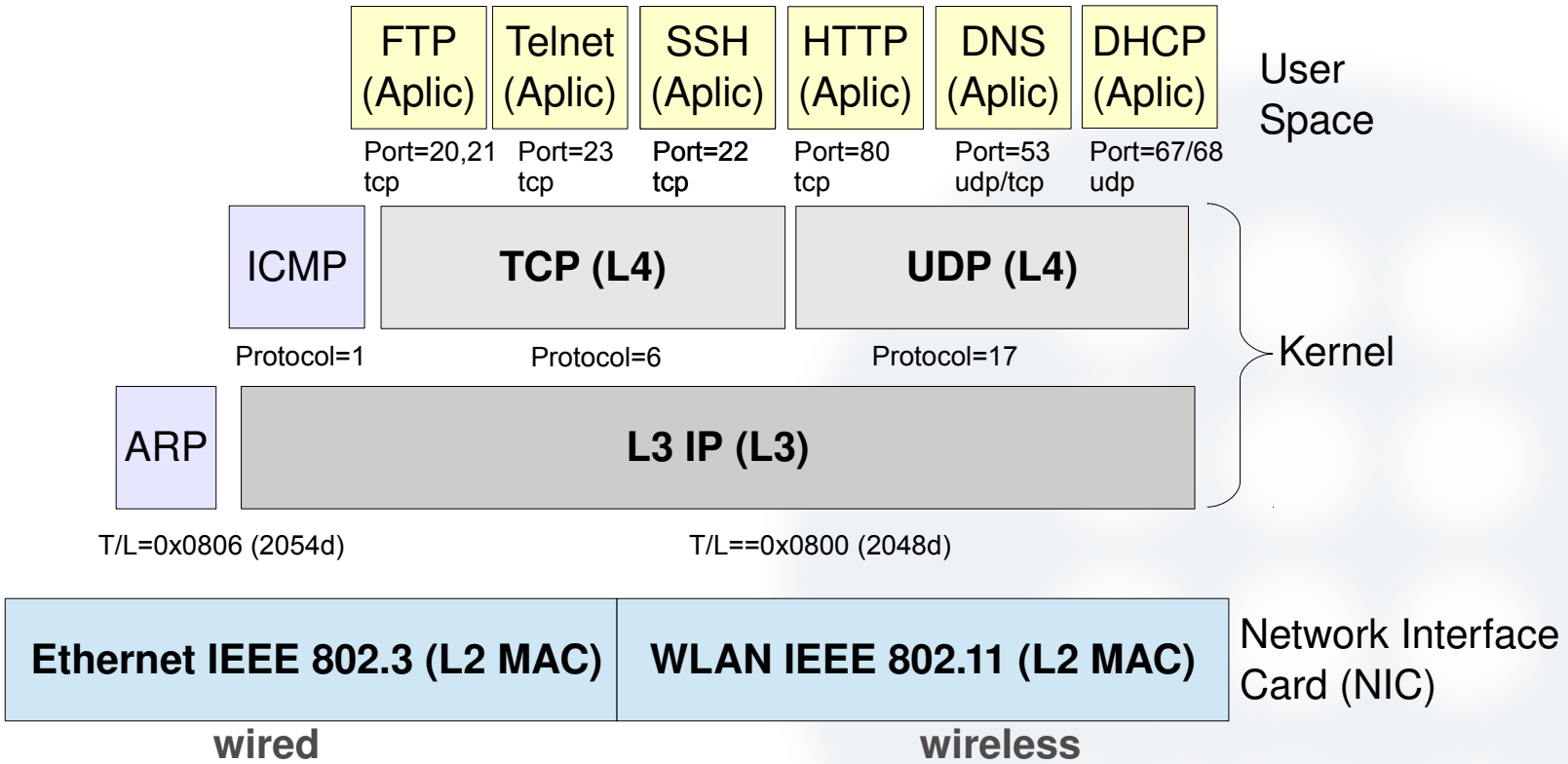
Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Basic Transport Protocols

- **User Datagram Protocol (UDP):**
  - UDP is the simplest transport protocol.
  - UDP is a message-oriented protocol (datagram protocol).
  - Each UDP datagram (message) is encapsulated over an IP datagram.
  - UDP only offers multiplexing capability (using ports) and a checksum for discarding wrong data for its users.
  - UDP does not provide error, flow or congestion control.
- **Transport Control Protocol (TCP):**
  - TCP provides applications with a full-duplex communication, encapsulating its data over IP datagrams.
  - TCP communication is connection-oriented because there is a handshake of three messages before data can be sent.
  - The TCP communication is managed as a data flow (TCP is not message-oriented).
  - Apart from multiplexing capabilities, TCP is a reliable protocol because it adds support to detect errors or lost data and retransmit them (**ARQ end-to-end error control**).
  - TCP also supports **an end-to-end flow control** and a **congestion control**.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# TCP/IP Protocol Stack



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro
- Transport Layer Motivation
- Client/Server Model
- Socket API
- netstat
- netcat

2 UDP

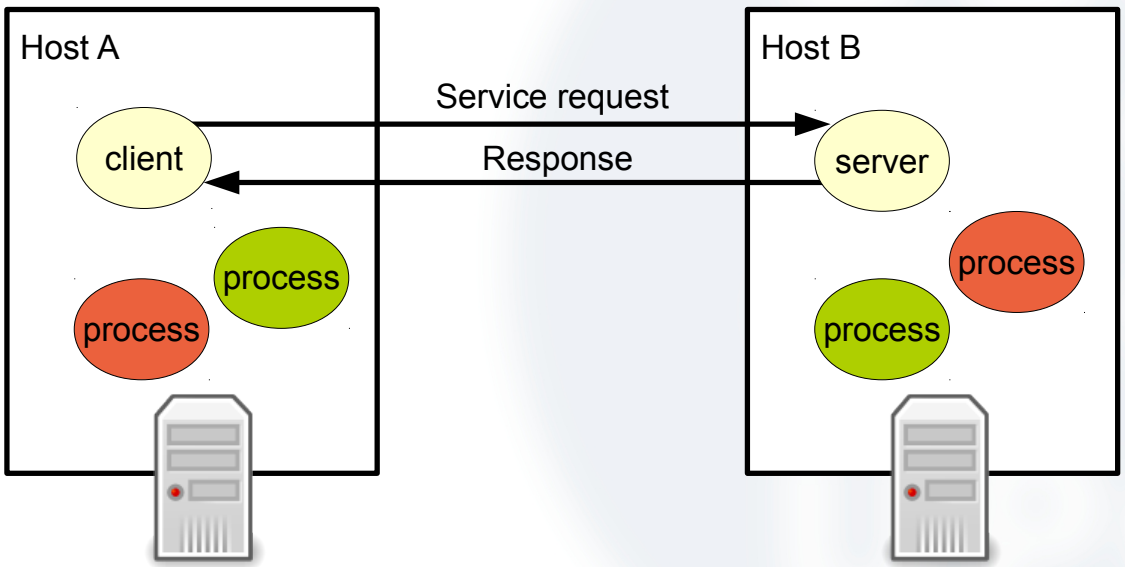
3 TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Client/Server Model I

- The client/server model is the most widely used model for communication between processes.
- Clients make requests to servers.
- Servers respond and they can generally support numerous clients.



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Client/Server Model II

- In Unix-systems, server processes are also called daemons.
- In general, a daemon is a process that runs in the background, rather than under the direct control of a user.
- Typically daemons have names that end with the letter ”d“ (e.g. telnetd, ftpd, httpd or sshd).
- **Clients initiate the interprocess communication,** so they must know the address of server.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

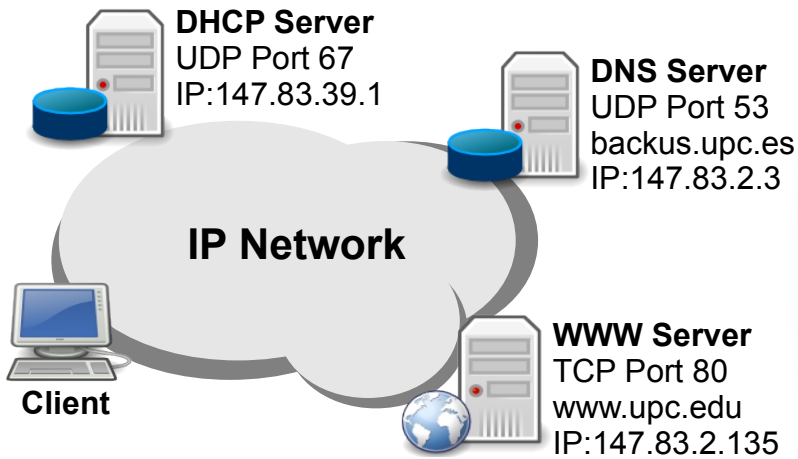
# Client/Server TCP/IP

- In the TCP/IP domain, the address of a process is composed of:
  1. An identifier called **IP address** (@IP) that allows reaching the destination "user space" or host in which the server process is running.
  2. An identifier of the process called **transport port**.
  3. The **transport protocol** used.
- So, the client process needs to know these three parameters to establish a TCP/IP socket with a network daemon (server).
- Internet uses a scheme called: well-known services or **well-known ports**.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

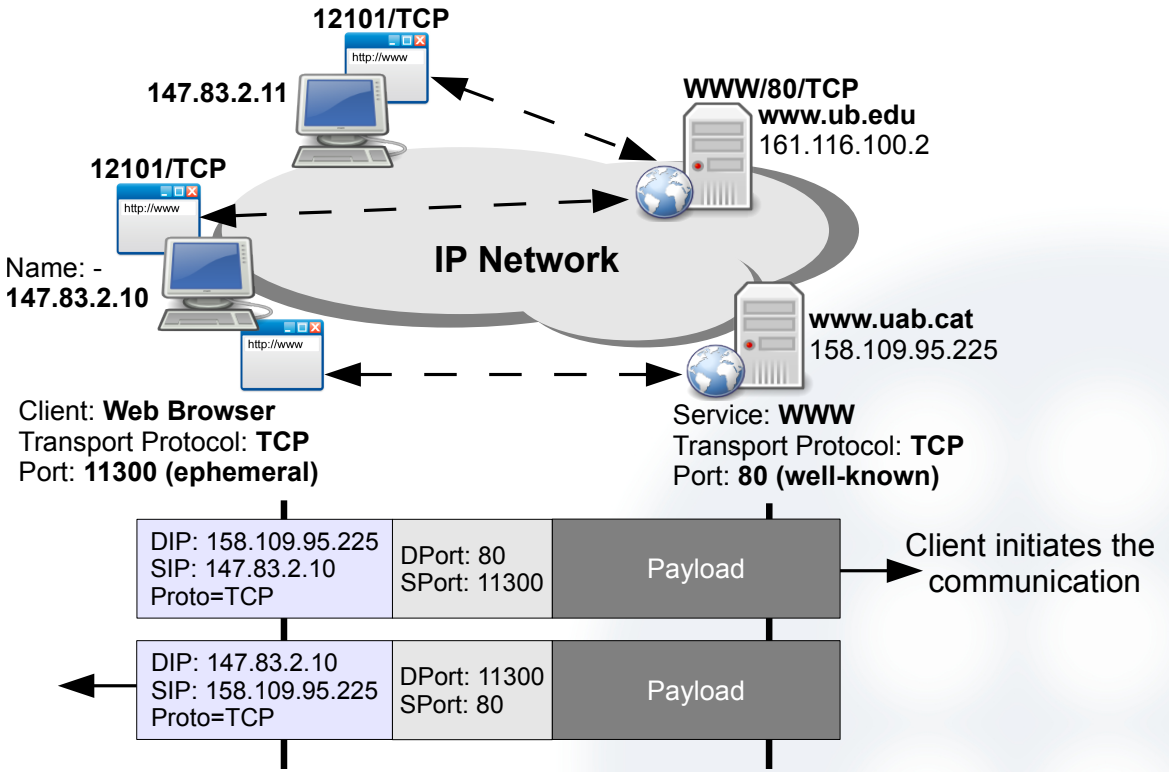
# Well-known Ports I

- The client needs to know IP, protocol and port to create a ”socket“ (communication) with a server.
- The client usually knows the IP (or **name**) of the server and there is a well-known transport protocol and port per service (determined by the application used).




- For example, HTTP servers (for the Web) use TCP/80, DNS servers use UDP/53 for name queries and the DHCP servers use UDP/67.
- Example clients: `host` command (DNS), `firefox` (DNS and HTTP), etc.

# Well-known Ports II




- Servers can manage multiple clients, identified by different `@IP/L4_Proto/Port` tuples.
- Typically, each client process asks for a free port to its OS kernel (ephemeral SPort).

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro
- Transport Layer Motivation
- Client/Server Model
- Socket API
- netstat
- netcat

2 UDP

3 TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Socket API I

- TCP/IP communications where developed in the context of Unix systems.
- One of the main ways of implementing TCP/IP communications is to use the ”socket“ API.
- An application programming interface (API) is an interface implemented by a software program to enable interaction with other software.
- Usually, an API is presented as set of functions collected in a library (C/C++ library).
- It may include specifications for routines, data structures, object classes and protocols used to communicate between the consumer and implementer of the API.
- **In Unix-like systems, sockets are the default API implemented by the kernel for providing an interface to networks to user-space processes.**

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Socket API II

- A socket is an endpoint of a bidirectional inter-process communication.
- The sockets API implemented in the kernel forbids two user-space processes to choose the same socket (`L4_Proto/Port`).
- With the socket API we can create TCP or UDP network sockets as client or server.
- By now, we will simplify this issue saying that:
  - Servers open sockets for ”listening“ for clients.
  - Clients open sockets for connecting to servers.
- In general, a server listens for traffic PDUs comming from ”any” interface (but system calls also let you select a particular interface or interfaces).
- The system calls for servers also allow to serve multiple clients simultaneously (multi-client server).

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

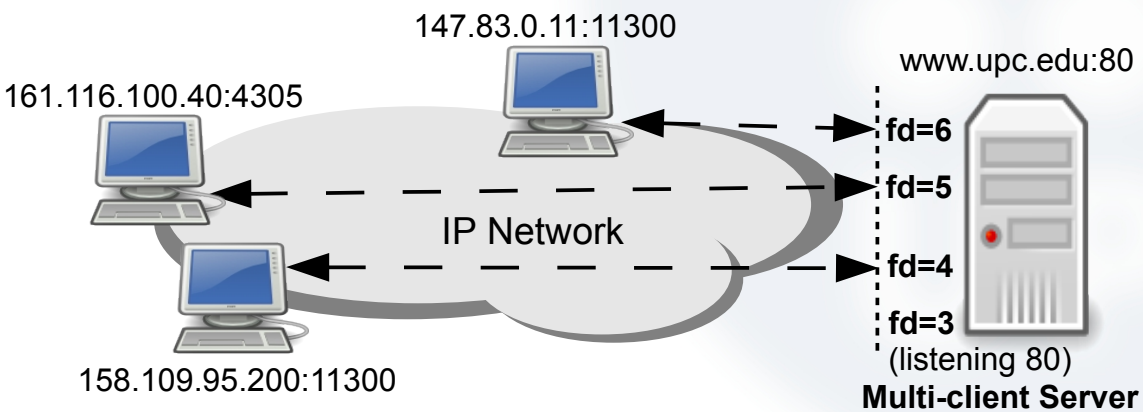
# Socket API III

- One of the main system calls is *socket()*, which returns a file descriptor.
- File descriptors of network communications receive the name of **socket descriptors** (*sd*).
- Socket descriptors are used to write (send) data to the ”network“ and to read (receive) data from the ”network“.
- As ”network“, we are referring to TCP and UDP communications.
- Unix Sockets.
  - In Unix-like systems, we have also Unix sockets.
  - Unix sockets are similar to TCP/IP sockets but they are local to the system [1].
  - They use filenames as addresses instead of tuples of network parameters (`@IP/L4_Proto/Port`).

[1] A detailed description of these sockets is beyond the scope of this document.

# Multi-client Servers

- Multi-client servers can serve several clients simultaneously.
- They work as follows:
  - The server creates a socket using the desired port in listening state.
  - The kernel returns the associated file descriptor (in our example fd=3).
  - Then, the server reads service requests through this fd.
  - For each new service request, clients are distinguished by `@IP/L4_Proto/Port`.
  - Finally, for each new client, the server creates a new socket (in our example fds 4,5 and 6).



# `lsof` and Sockets

- Recall that the `lsof` command shows us the "list of open files" (including socket descriptors).
- Example:

```
alice:~# $ lsof -a -p 4578 -d0-10
COMMAND PID    USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
nc      4578   user1    0u   CHR  136,2      0t0    5 /dev/pts/2
nc      4578   user1    1u   CHR  136,2      0t0    5 /dev/pts/2
nc      4578   user1    2u   CHR  136,2      0t0    5 /dev/pts/2
nc      4578   user1    3u  IPv4 149667      0t0  TCP localhost:
48911->localhost:12345(ESTABLISHED)
```

- The previous command is used to view the file descriptors of the process with PID 4578.
- We see that the fd=3 is associated with an established TCP connection.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro
- Transport Layer Motivation
- Client/Server Model
- Socket API
- netstat
- netcat

2 UDP

3 TCP



# netstat

- The command `netstat` (network statistics) shows established or listening sockets and several related statistics.
- Options:

| | |
|---|---|
| -t | TCP connections. |
| -u | UDP connections. |
| -l | listening sockets. |
| -n | addresses and port numbers are expressed numerically and no attempt is made to determine names. |
| -p | show which processes are using which sockets (you must be root to do this). |
| -r | contents of the IP routing table. |
| -i | displays network interfaces and their statistics. |
| -c | continuous display. |
| -v | verbose display. |
| -h | displays help at the command prompt. |

- Example:

```
alice:~# netstat  -tnlp
Active Internet connections (only servers)
Proto Local Address  Foreign Address  State    PID/Program name
tcp   0 0.0.0.0:80   0.0.0.0:*        LISTEN   1690/apache2
tcp   0 :::22        :::*             LISTEN   1037/sshd
```

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1. Net Apps Intro
   - Transport Layer Motivation
   - Client/Server Model
   - Socket API
   - netstat
   - netcat
2. UDP
3. TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# netcat I

- The `netcat` application can be used to create a process that opens a raw TCP or UDP socket as client or server.
- It is very useful tool for testing networks (known as “Swiss Army Knife of networking“).
- Note. Make sure that your `netcat` command is the “traditional” one.
- `netcat` as client:

```
$ nc hostname port
```

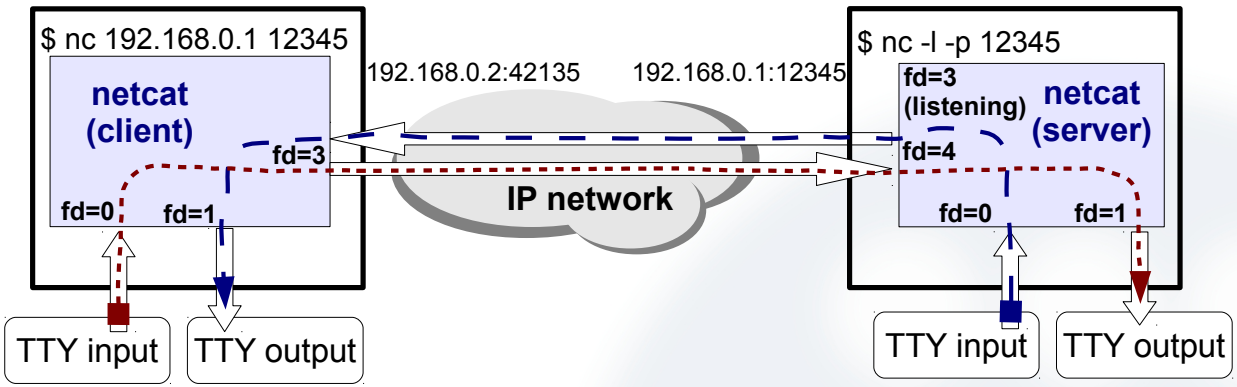
- We can use the -l (listening) option to make `netcat` work as a server:

```
$ nc -l -p port
```

- A server with `netcat` **is not multi-client**.

# netcat II

- Once a client is connected, the behavior of `netcat` until it dies (e.g. CRL+c) is as follows.




  - The `netcat` processes read data from stdin (fd=0) and write these data to fd=3 in the client or to fd=4 in the server.
  - On the other hand, the data received from the network is read from fd=3 (client) or fd=4 (server) and then, written to fd=1 (stdout).
  - Note. To implement `netcat` we can use the C system calls `write()` and `read()`, these system calls use as parameter the fd.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# netcat III

- Options of `netcat` (traditional):

| | |
|---|---|
| -h | show help. |
| -l | listening or server mode (waiting for incoming client connections). |
| -p port | local port. |
| -u | UDP mode. |
| -e `cmd` | execute `cmd` after the client connects.. |
| -v | verbose debugging (more with -vv). |
| -q secs | quit after EOF on stdin and delay of secs. |
| -w secs | timeout for connects and final net reads. |

- Transfer files:

```
$ cat file.txt | nc -l -p 12345 -q 0
```

- Create a remote terminal:

```
$ nc -l -p 12345 -e /bin/bash
```

- Echo server with `netcat` (only one client at one time).

# netcat IV

```
1 #!/bin/bash
2 # nc-echo.sh
3 while true
4 do
5 nc -l -p 12345 -e /bin/cat
6 done
```

- Execute client and server and take a look at open files.
- View established connections:

```
$ netstat -tnp
Active Internet connections (w/o servers)
Proto Local Address    Foreign Address  State       PID/Program name
tcp   127.0.0.1:41426  127.0.0.1:12345  ESTABLISHED 14688/nc
tcp   127.0.0.1:12345  127.0.0.1:41426  ESTABLISHED 14687/cat
```

- View open files of `cat`:

```
$ lsof -a -p 14687 -d0-10
COMMAND PID   USER FD TYPE DEVICE NAME
cat     14687 user 0u IPv4 39942  TCP localhost:12345->localhost:41426
cat     14687 user 1u IPv4 39942  TCP localhost:12345->localhost:41426
cat     14687 user 2u CHR  136,6  /dev/pts/6
```

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP

3 TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP
- UDP Datagram
- UDP Applications

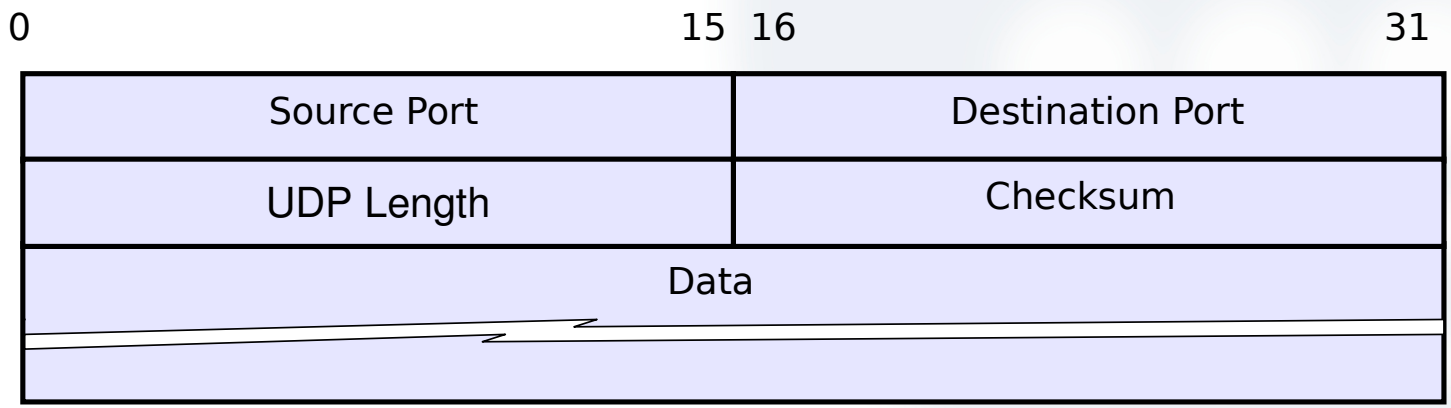
3 TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# UDP Basics

- User Datagram Protocol (RFC 768).
- An UDP packet or **UDP datagram** is considered an extension of IP.
- An UDP datagram is like an IP datagram with the capability of multiplexing and demultiplexing transport communications.
- Best effort (like IP) because UDP messages may be lost or corrupted.



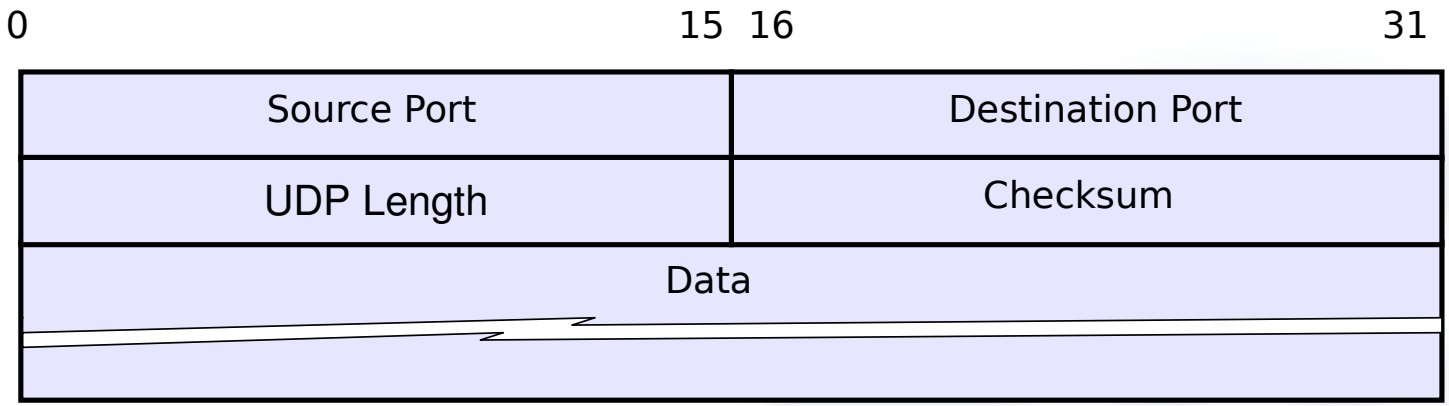
Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# UDP Ports

- Transport communications are communications between processes.
- A port is used to identify each process at each host.
- The port ID has a size of 16 bits.
- The source port is optional in UDP.
- If the source UDP port is not used, it is set to zero.
- If a reply is required, a source UDP port must be used.
- In this last case, the port is usually dynamically assigned (ephemeral port) and typically, a value above 1024 is used.
- Note. The set of UDP ports is independent of the set of TCP ports.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

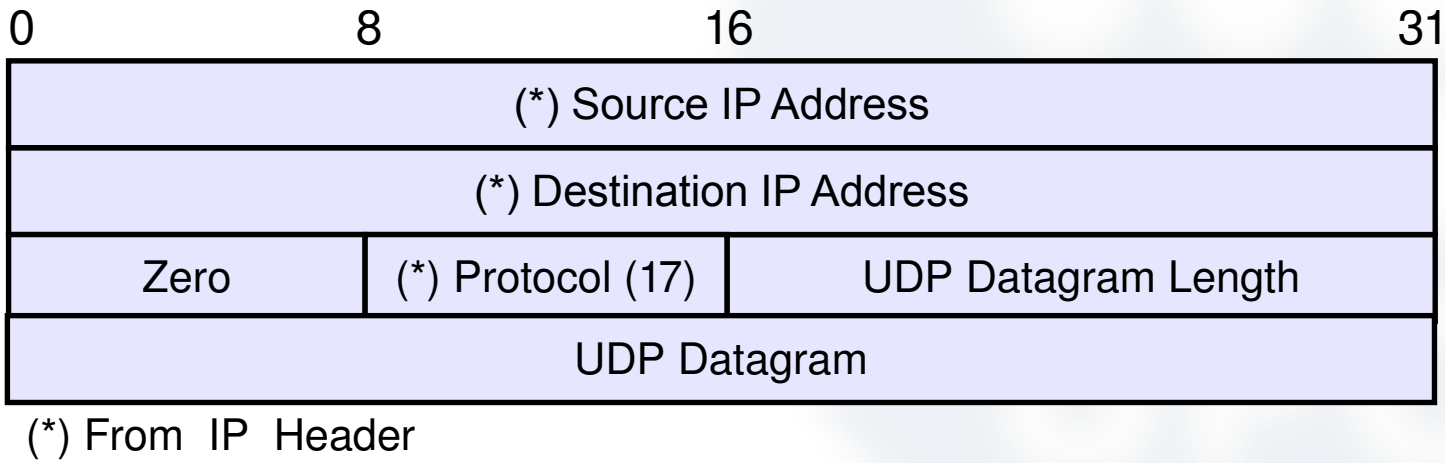
# UDP Datagram




- UDP Length:
  - Includes header and data (in bytes).
  - Minimum value is 8, which is the minimum header length.

# Checksum

- UDP checksum:
  - Optional. If unused, it is set to 0.
  - Checks the integrity of the UDP header, UDP payload data, and IP routing.
  - A pseudo-header is used so the receiver can make sure that no routing errors have occurred.
  - If a wrong UDP datagram is received, it is silently discarded.




(*) From IP Header

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP
- UDP Datagram
- UDP Applications

3 TCP

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# UDP Service

- UDP service is message-oriented.
- The transmission of each UDP datagram (UDP PDU) is independent.
- The transmission of an UDP datagram works as follows:
  - An UDP process (UDP user) delivers a message (payload) to the UDP layer.
  - The UDP layer, which is implemented in the kernel of the system, adds the UDP header of 8 bytes to the payload.
  - If the UDP PDU (payload+UDP_header) is larger than the MTU, fragmentation is performed at IP level.
  - The largest possible size of an UDP datagram is: $2^{16} - 1 - 20 - 8 = 65.507$.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# What UDP Does not Offer

- As explained, UDP is not connection-oriented.
- It does not offer:
  - **Sequencing**: does not provide numbering of UDP datagrams.
  - **Error Control**: no recognition, neither positive nor negative. However, we can discard an UDP datagram if the checksum is used and it is not correct.
  - **Flow Control**: there can be overflow in the receiver.
  - **Congestion Control**: messages can collapse the network.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Advantages of Datagrams

- No delay due to connection establishment.
- Simple transmitters and receivers.
- It uses minimal system resources in terms of memory and CPU.
- Adds a small header (8 bytes).
- Since it does not use any congestion control mechanism, the application can define any desired pattern to transmit its data flows.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# UDP Applications

- At the transport layer we have reliability problems like losses, disorders, duplications, delays, etc.
- TCP provides a solution for the previous problems.
- Many applications that do not want to manage these problems use TCP.
- Actually, the majority of the Internet applications use TCP[2].
- Applications that do not want the management of these problems like TCP does, use UDP and implement their own solutions.
- Typical UDP applications are:
  - Real-time services like voice or video.
  - Request/response services like DNS.
  - Simplex services (one way).
  - Multicast applications (which are not implementable with TCP).

[2] This is why this architecture is called TCP/IP... but UDP exists!

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP

3 TCP



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP

3 TCP
- Connection Management
- Flow Control
- Error Control
- TCP segment

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Introduction to TCP

**Goal of TCP:** provide a reliable full-duplex transport of a flow of bytes between two processes.

- **Reliable**: TCP delivers the same sequence of bytes that were delivered by the source to the destination.
- **Full-duplex**: there is a concurrent transfer in both directions.

**Fundamental Mechanisms**:

- **Flow Control** (original RFC 793).
- **Error Control** (original RFC 793 with enhancements).
- **Congestion Control** (many implementations after the congestion crash of 1986).

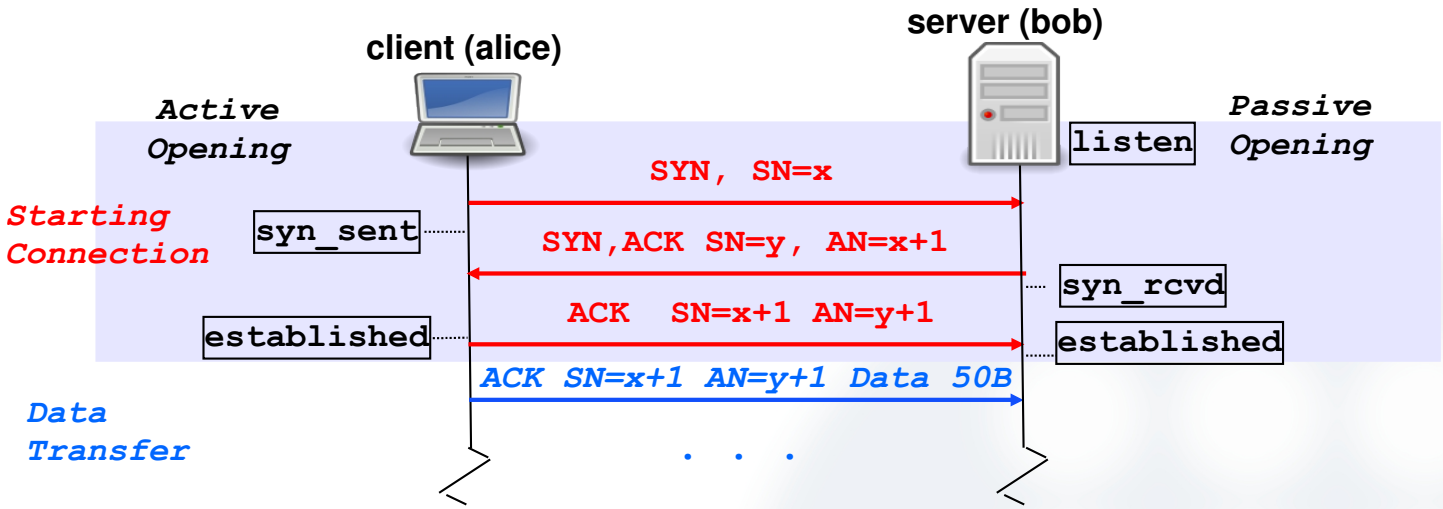
TCP is suitable for large data volumes transfer where timing is not critical.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Connection Oriented

- **Establishment**:
  - Handshake of three segments initiated by the client.
  - At the end of this stage both ends:
    - Have agreed on a certain initial sequence number for each direction.
    - Know the ports used by each other.
  - After the establishment, TCP ignores any attempt to establish again the same connection.
- **Transmission**:
  - At the transmission stage both ends are equal.
  - At this stage both ends can send information through the TCP connection.
- **Release**:
  - If one end does not have any more data to send it can close its way of the communication.
  - The connection is close when the two ends close.

# Connection Establishment I




- The connection is established with 3 TCP segments.
- This is also called TCP **3-way handshake**.
- The **first** segment has in its TCP header:
  - A flag called SYN activated.
  - The initial sequence number (SN) randomly chosen to number the bytes of the flow that is transmitted from client to the server (in our example is $x$).

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Connection Establishment II

- The **second** segment has in its TCP header:
  - The flags SYN and ACK activated.
  - The initial Sequence Number (SN) randomly chosen to number the bytes of the flow that is transmitted from server to the client (in our example is *y*).
  - The Acknowledgement Number (AN) set to the next byte that is expected to be received from the client.
  - Note that in the second segment AN=x+1.
  - This is because a segment with the flag SYN activated has to be acknowledged.
  - To do so, a SYN segment is considered the first byte of the flow.
- The **third** segment has in its TCP header:
  - The flag ACK activated.
  - The Acknowledgement Number (AN) set to the next byte that is expected to be received from the client.
  - Note that in this segment AN=y+1.

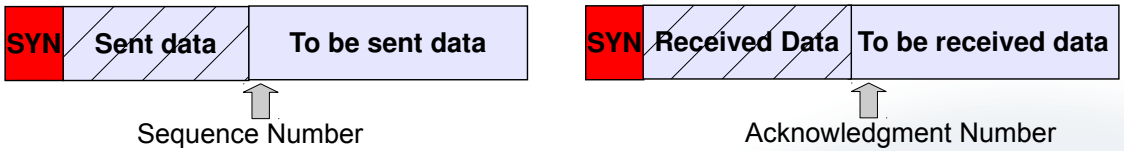
Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# TCP Numbering I

- The SN and AN have 32 bits and sequence numbers are reused cyclically (wrap around).
- **Sequence Number (SN):**
  - The SN indicates the position (byte number) of the data contained in the segment in the corresponding flow.
  - The SN is only increased for segments containing payload (data) since only bytes of data are numbered.
  - The only exceptions are segments with SYN or FIN activated.
  - The FIN flag is used for ending a TCP connection.
  - SYN and FIN count respectively, as the first and last bytes of the flow.

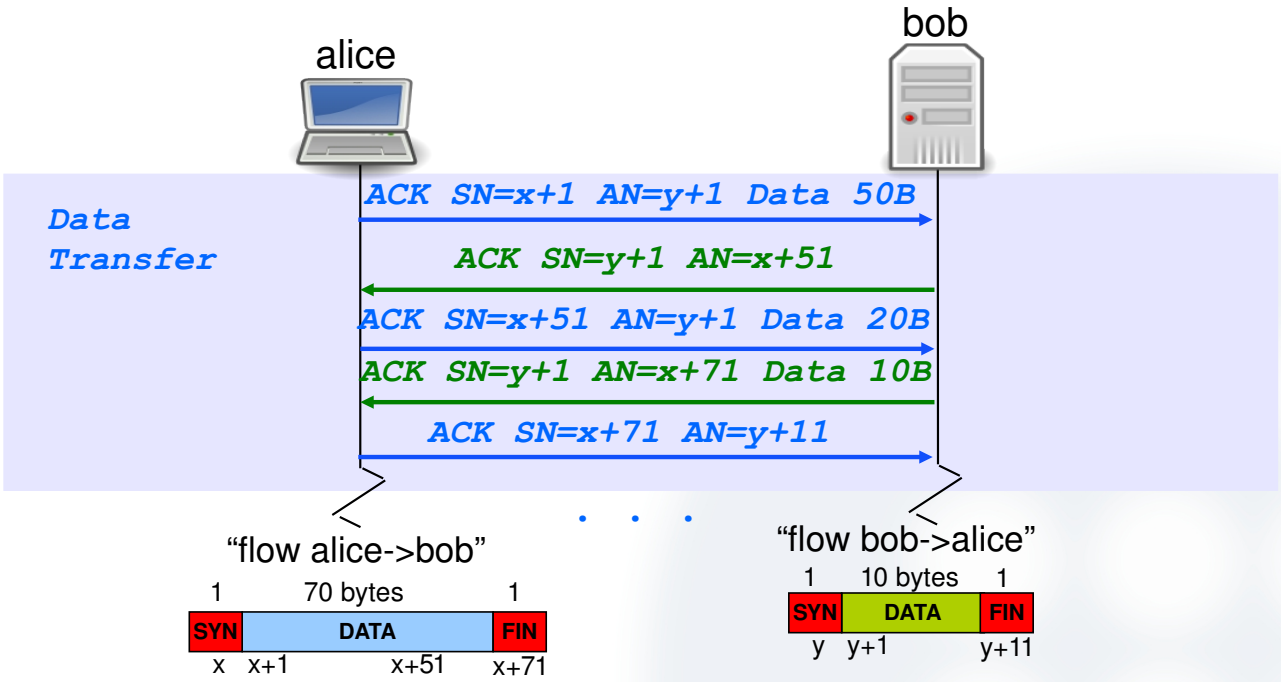
Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# TCP Numbering II




- **Acknowledgement Number (AN):**
  - The AN indicates the next byte number that the transmitter is expecting to receive.
  - This is a cumulative ACK scheme.
  - The AN allows piggy-backing in TCP and delayed ACKs are used to profit piggy-backing.
  - Basic TCP sends an ACK for each correctly received TCP segment.

# TCP Data Transmission



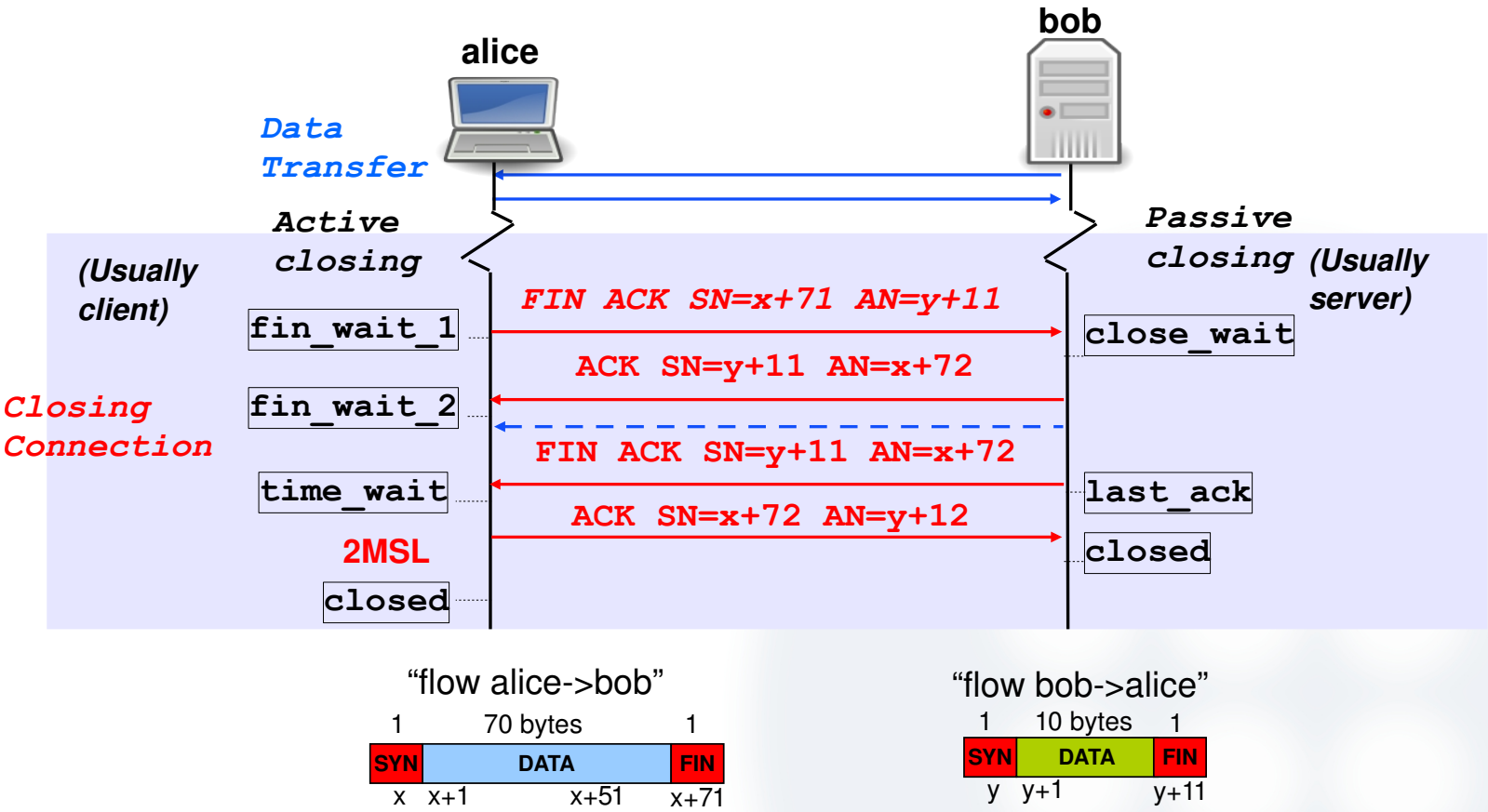

- In the previous example alice has sent to bob 70 bytes and bob has sent to alice 10 bytes.
- Note that bob was able to do one piggy-backing.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Connection Reset

- A connection is reseted activating a bit in the TCP header.
- The connection is reseted when there are unrecoverable errors.
- Examples:
  - An out of sequence segment is received (impossible sequence number).
  - A segment is received for a closed port.
- A reset causes both sides immediately to release the connection and all its resources.
- As a result, transfer of data ceases in both directions, which can result in loss of data that is in transit.
- A reset indicates an abnormal termination of the connection.

# Connection Release




- Each way can be closed independently.
- If only one way is closed, this is called a "half-close".
- The release of a connection can be done with 3 or 4 TCP segments.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Maximum Segment Life I

- The end that actively closes the connection remains in TIME_WAIT state for 2MSL (Maximum Segment Life).
- 2MSL is typically set to around 1 minute and it is the worse possible Round Trip Time.
- During the 2MSL the TCP port is occupied and **no other process can use it**.
- After this time, the end that performs the active close goes to the CLOSED state and clears the connection records.

**Why wait for the 2MSL?**

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Maximum Segment Life II

- TCP responds with a RESET (segment with the RESET flag activated) if a segment for an inexistent connection is received.
- While closing the TCP connection, the last ACK may be lost.
- If the last ACK does not reach the destination, the FIN is retransmitted.
- In this case, an ACK for the FIN must be generated but if the connection is closed, a RESET is generated instead of the ACK.
- The end of the active closing waits 2MSL to make sure that its last ACK has been received.

# TCP Finite-State Machine

**CLOSED** State of connection

**LISTEN** Primitive service executed on the server

**CONNECT** Service primitive executed on the client

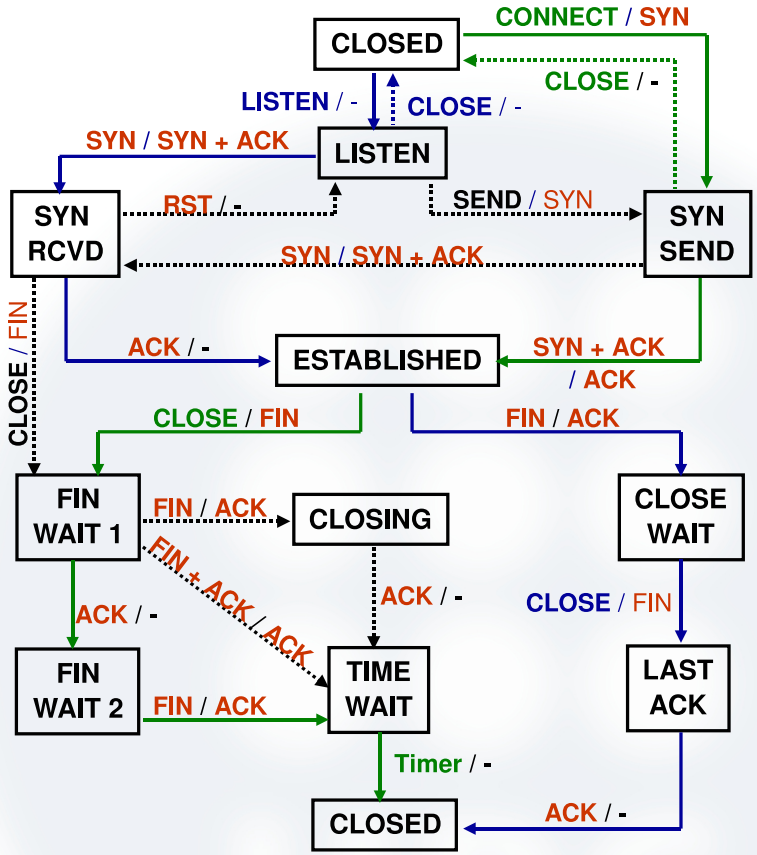
**SYN** Flag in the sent or received segment

Normal transition in the server

Normal transition in the client

Unusual transition

**X/Y** X: Event that causes the transition
Y: resulting action

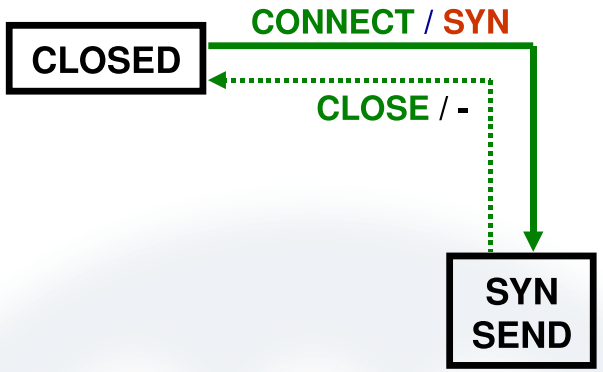


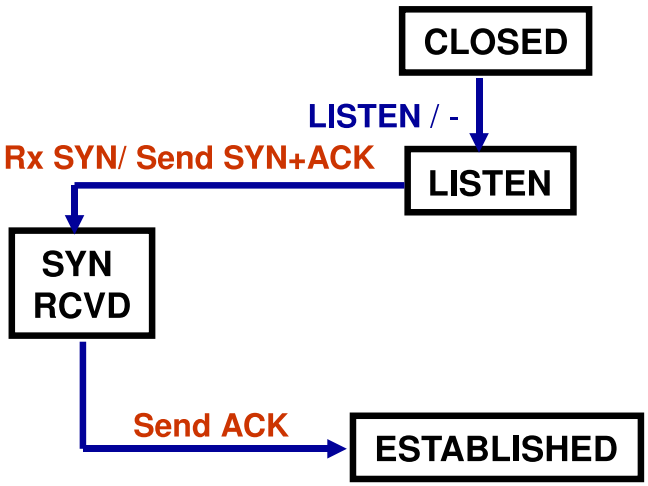
Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Connection Establishment

Client:

- Begins the connection establishment (active open).
- Sends a segment with the SYN flag activated.



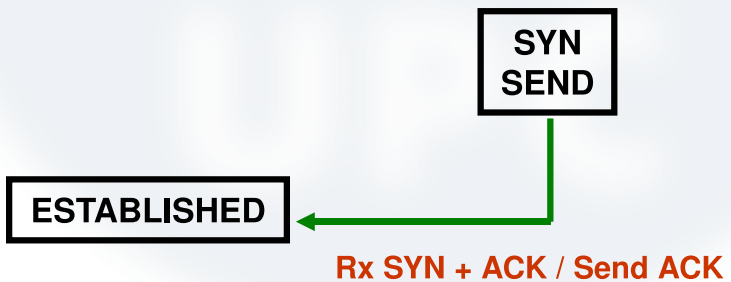




Server:

- It is listening and receives a connection request (passive open).
- It sends a segment with the SYN and ACK flags activated to the client.

Client:

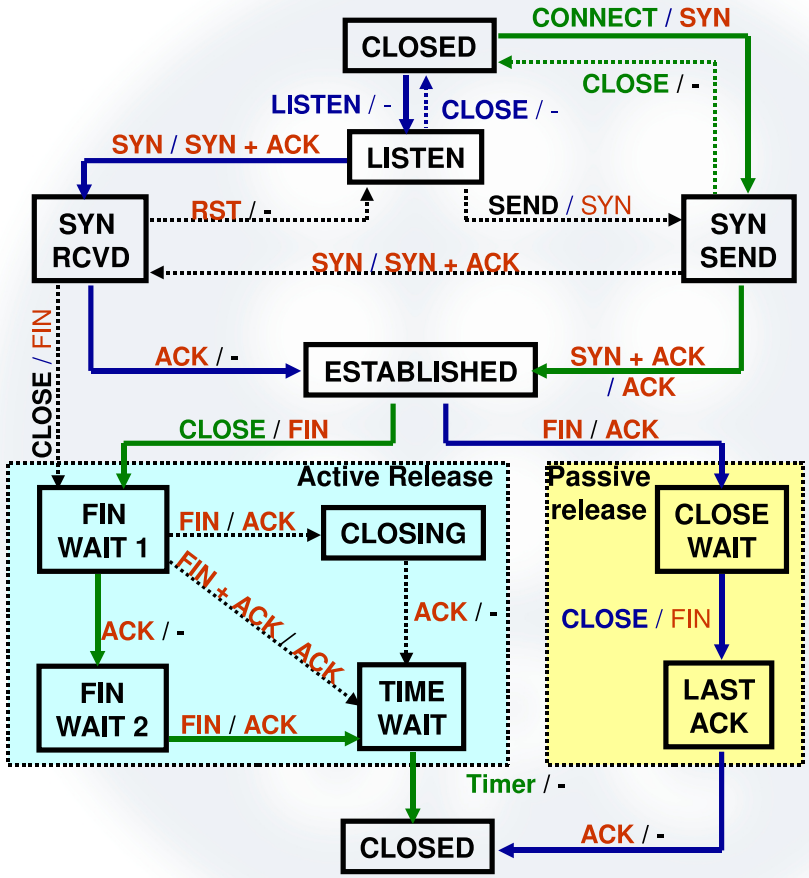
- Final segment of the 3-way handshake with the ACK flag activated.



# Types of Releases

- **Active Release**: release by the local end.
- **Passive Release**: release by remote end.

- **Abrupt release**:
  - Occurs whenever there is a problem with the channel or with one of the ends.
  - It is detected with timers.
  - The transitions of this case are not displayed in the state diagram.



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Overiew of Basic TCP

- **TCP Connection Management.** $\surd$
- **Flow Control:**
  - Sliding Window (original).
  - Nagle algorithm (Tahoe).
- **Error Control**:
  - GobackN (original)
  - Delayed ACKs (enhancement of RENO)
  - Improved calculation of the RTO using the algorithm of Jacobson & Karn's (enhancement of Tahoe)
  - Fast Retransmit (enhancement of Tahoe)
- **TCP segment format**.
- **Congestion Control**:
  - Slow-start (Tahoe)
  - AIMD congestion avoidance (Tahoe)
  - Fast Recovery (Reno)

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Original TCP

- The original TCP was defined in the RFC793.
- Goals:
  - Provide a reliable byte stream (ordered and error free) between two processes using a TCP connection.
  - Check that the flows in each way do not overload the receivers.
- For this purpose:
  - Defines the connection management.
  - Defines a flow control based on a sliding window.
  - Defines an error control based on a Go-back-N.
- It does not consider network congestion.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP

3 TCP
- Connection Management
- Flow Control
- Error Control
- TCP segment



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Sliding window I

- TCP uses a sliding window as flow control mechanism.
- ACKs are used simultaneously by the error control and the flow control to minimize the exchange of control information.
- The receiver sends an ACK for each TCP segment received correctly.

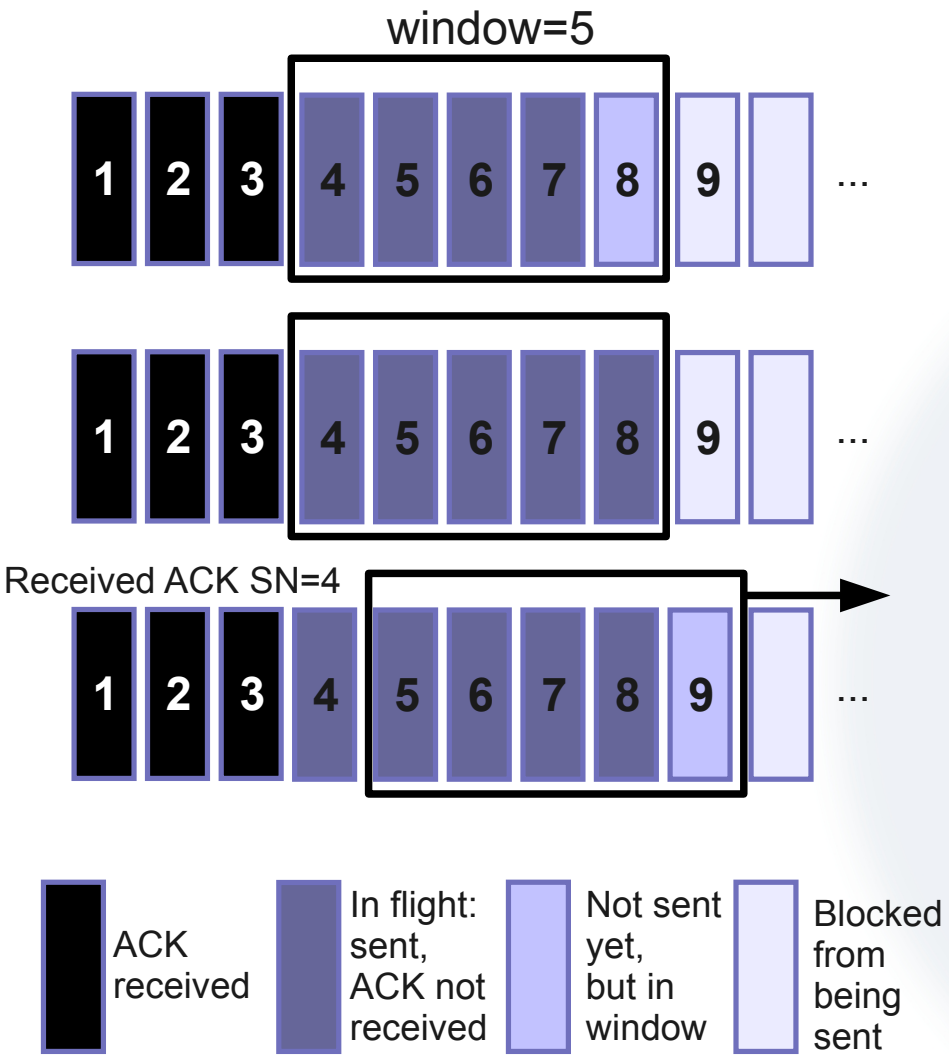
## TCP window is byte-oriented

The TCP window is defined as the maximum number of unacknowledged bytes (bytes for which an not frames without ACK) that the sender can have at any time.

- The ACK indicates the next byte to be received (cumulative scheme).
- Unacknowledged bytes are said to be “in-flight“.
- The window size is set by the receiver based on its reception buffer availability (called ”**AdvertisedWindow**“).

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Sliding window II




- In-flight bytes are stored in a queue by the sender until their corresponding ACK arrives.
- After an ACK is received, the sender can transmit the new data allowed by the window (sliding window effect).

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Filling the Pipe I

- How this mechanism can be used to manage the flow rate?
  - Varying the window size the receiver can adjust the throughput (flow rate) properly.
- Which is the resulting rate?
  - If RTT (Round Trip Time) is the time that takes a segment to reach the receiver and the sender to receive the corresponding ACK.
  - Then, the source rate $S$ will be:

$$S = wnd/RTT[bps]$$

  - With $S \leq B_{tx}$, where $B_{tx}$ is the channel transmission rate.
  - Let's see this graphically.

# Filling the Pipe II

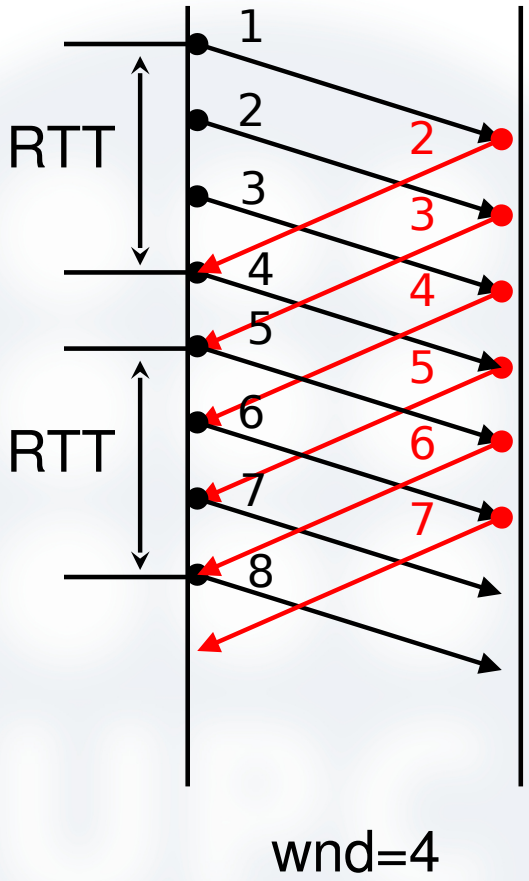
**”Law Of Conservation Of The Packet“**

If we want to fill the transmission ”pipe” as much as possible:
**We must be allowed to send a new bytes when the ACK of the first segment of the window arrives.**

- In this case, *wnd* must be:

$$wnd \geq B_{tx} \times RTT \text{ [bytes]}$$

- Bandwidth-Delay-Product.
  $BDP = B_{tx} \times RTT$
- Source rate:
  $S = min(B_{tx}, \frac{wnd}{RTT})$ [bps]



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Maximum Segment Size I

- Hosts are not required to reassemble infinitely large IP datagrams.
- The Maximum Segment Size (MSS) specifies the maximum size of TCP payload that a host can receive in a single TCP segment (complete or in fragments).
- By default, the maximum size IP datagram that all hosts are required to accept or reassemble from fragments is 576 octets:
  - A datagram of 576 implies a MSS=576-20-20=536 bytes.
  - The previous calculus is done using the minimum IP and TCP headers.
  - When TCP is used in a situation where either the IP or TCP headers are not minimum, the sender must reduce the amount of TCP payload in any given packet by the number of octets used by the IP and TCP options.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Maximum Segment Size II

- Note that the default MSS provides poor performance in networks in which we can send larger datagrams.
- For this reason:
  - TCP provides an option that may be used at the time a connection is established to indicate the MSS that can be accepted on that connection.
  - This Maximum Segment Size (MSS) announcement is sent from the data receiver to the data sender.
  - Says “I can accept TCP segments up to size X”.
  - Each direction of data flow can use a different MSS.
  - Typically, the MSS option is established by the OS Kernel using the output interface MTU.
  - For example, for Ethernet MTU = 1500, then MSS = 1500-20-20= 1460 bytes.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Silly Window Syndrome (SWS)

- Flow controls based on sliding windows can suffer from the “silly window syndrome”.
- This syndrome makes the sender to transmit small segments (tinygrams) instead of the largest possible segments of size MSS.
- Extreme Tinygram:
  - 1 byte of payload data.
  - 40 bytes of headers (TCP+IP)!!
- The syndrome can be caused by the receiver or by the transmitter:
  - Because the **receiver** announces small window sizes.
  - Because the **transmitter** sends only small amounts of data.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Avoiding SWS on the Receiver

- The idea is that the receiver must avoid announcing small windows.
- The algorithm to avoid the SWS at the receiver side says that given an **MSS** and a receiver buffer size of **Q**:
  - The receiver can announce larger windows only if it can increase the window by an **MSS** or by **Q/2**, whichever is less.
- In other words, we restrict the receiver from increasing the window by too small an amount.
- The receiver can only increase the window if she has MSS bytes free or or one-half of the buffer free, whichever is less.
- Typically MSS is less than one-half of the buffer in current systems.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Avoiding SWS on the Sender I

## Nagle algorithm

- The idea of the Nagle algorithm is combining a number of small outgoing messages, and sending them all at once.
- In particular, the sender is not allowed to transmit data unless:
  - All his sent data has been recognized.
  - There is enough data in his TCP output buffer to fill a segment MSS-sized.
- In pseudo-code:

```
1  if there is new data to be sent
2    if the window_available >= MSS and available data is >= MSS
3      send complete MSS segment now
4    else
5      if there is still unconfirmed data in the pipe
6      enqueue data in the buffer until an acknowledge is received
7      else
8        send data immediately
9      endif
10   endif
11 endif
```

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Avoiding SWS on the Sender II

## Nagle algorithm

- Nagle creates a kind of stop & wait transmission for transmitting tinygrams.
- For transmissions of large volumes of data, Nagle does not affect because we will always have an MSS available to send.
- Nagle minimizes the number of tinygrams (which are inefficient).
- Problems with some implementations:
  - The Nagle algorithm does not allow a situation in which we need to send datagrams with a small payload but we need little delay (like for protocols of display servers).
  - The algorithm can be disabled when creating the TCP socket with the option TCP_NODELAY.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Persist Timer

- The persist timer is necessary in a situation as follows:
  - The receiver has closed its window (AdvertisedWindow=0).
  - Later, there is buffer space free again and the receiver sends an ACK to open the window.
  - The ACK is lost and remember that ACKs are not retransmitted in TCP.
  - The receiver is expecting to receive data.
  - The transmitter is waiting that receiver opens the window.
  - As a result, the connection is blocked.
- Solution:
  - The transmitter periodically (each persist timer) queries the receiver to check if, in fact, the window has been increased (window probes).
  - The segment sent during a window probe has a payload of 1 byte.
  - The receiver does not acknowledge this byte while the window is 0.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP

3 TCP
- Connection Management
- Flow Control
- Error Control
- TCP segment

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

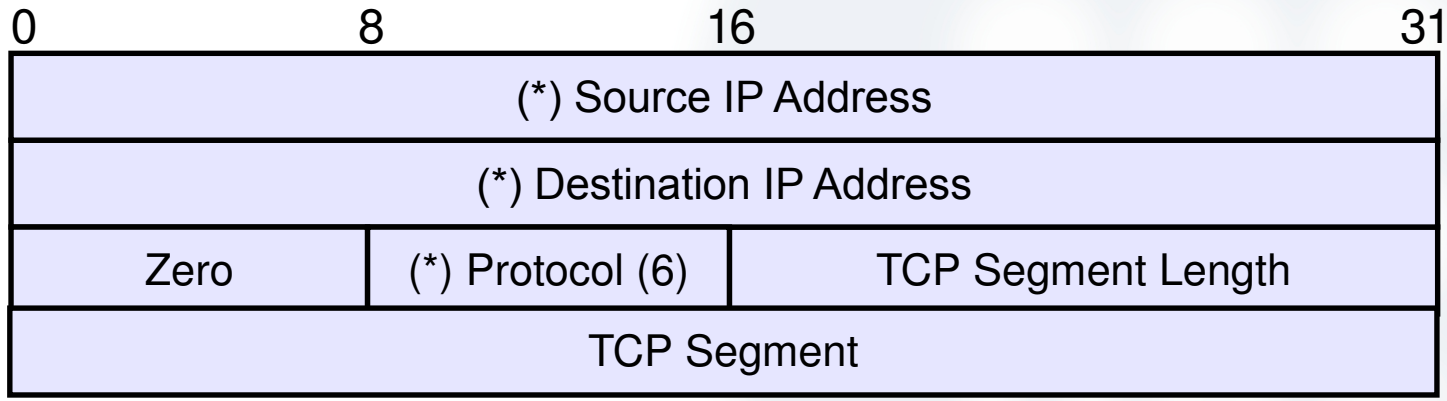
# Error Control

- Basic Error Control.
  - Described in RFC 793 (old TCP).
  - ARQ mechanism (a sort of Go-Back-N).
  - Loss detection (by timeout).
- Fast Retransmit (FR).
  - Rapid detection (by duplicate ACKs).
  - TCP Tahoe and later (defined in RFCs 2100 and 2581).

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

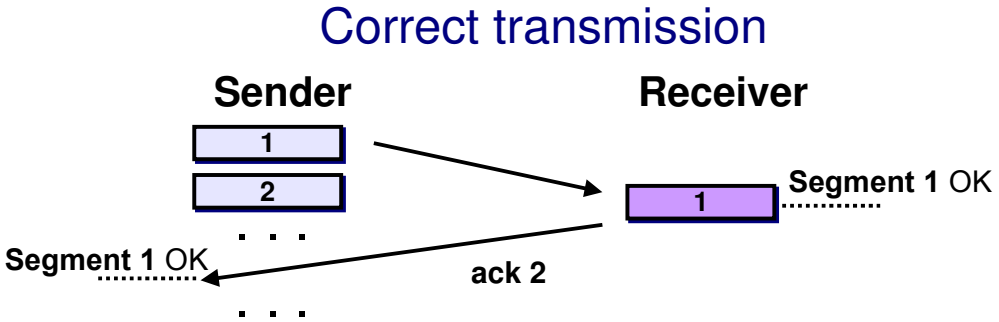
# Error Detection

- **TCP Checksum.**
  - TCP uses a checksum like the one of UDP.
  - TCP checksum is mandatory (UPD checksum is optional).
  - The checksum is computed using a pseudo-header that checks the integrity of the TCP header, TCP payload and IP routing.



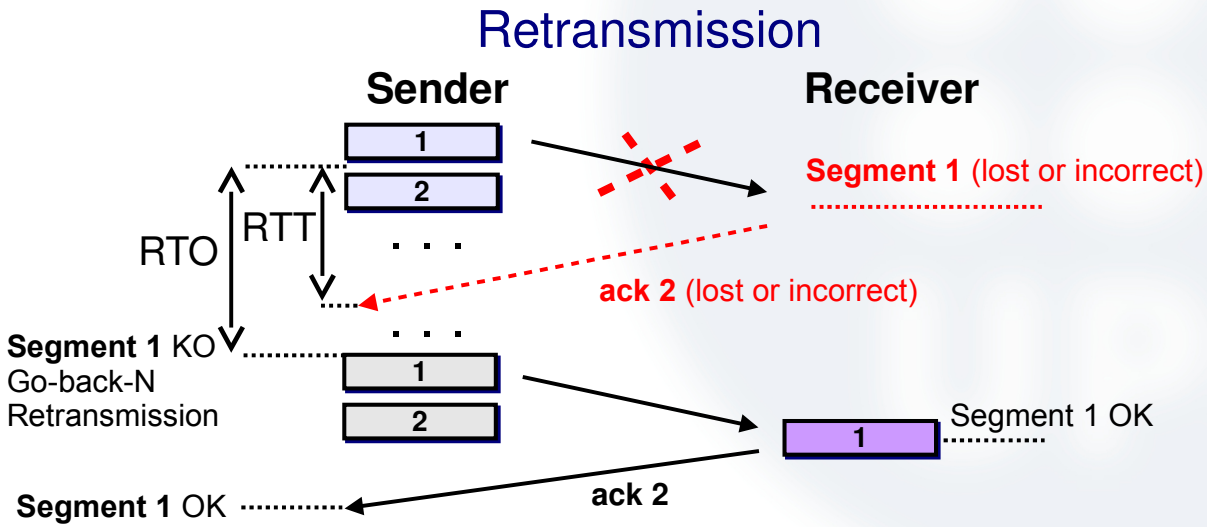

(*) From IP Header

# Basic Error Handling




- The sender starts a Retransmission Time Out (RTO) for each segment that it sends.

- If the RTO expires without any ACK received, the corresponding segment and all the subsequent segments are retransmitted (GoBackN).
- RTO may expire because the corresponding segment is lost or incorrect or because a serie of ACKs are lost or incorrect.

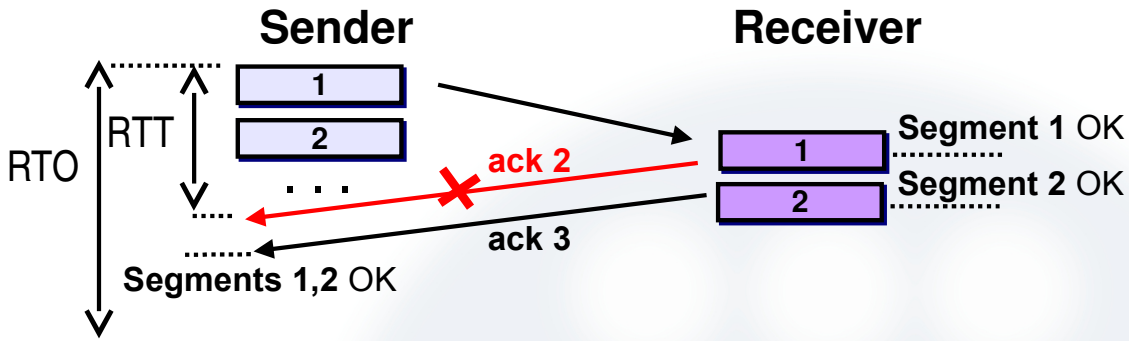


Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

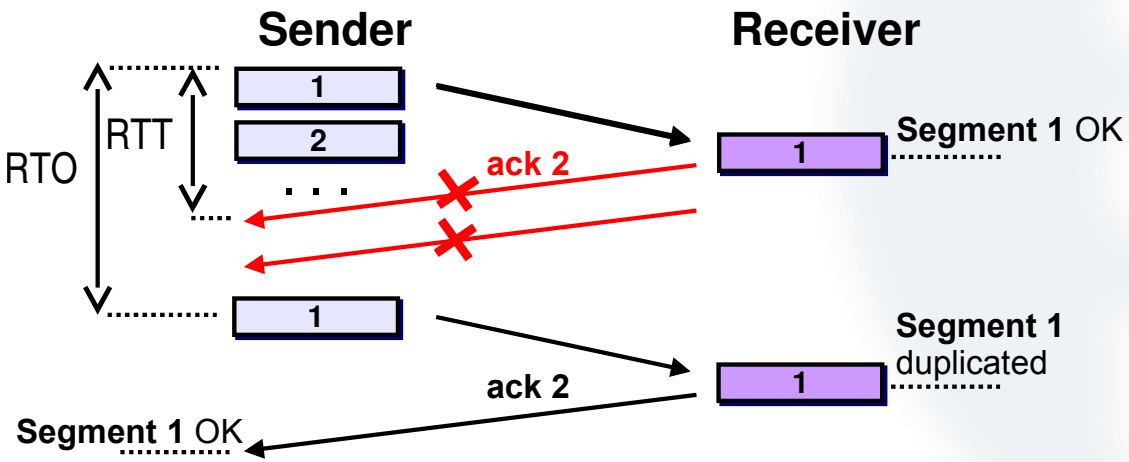
# Error Handling Cont. I

An ACK loss does not imply a retransmission.

## Cumulative ACKs



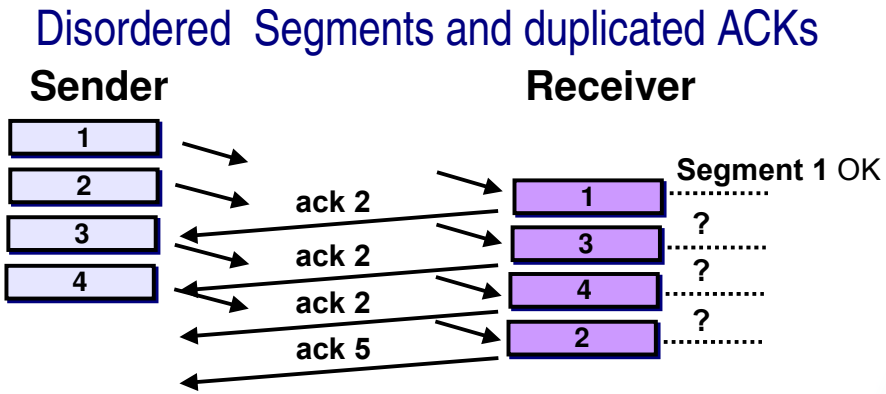

## Duplicate Segments




There can be duplicated segments.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Error Handling Cont. II

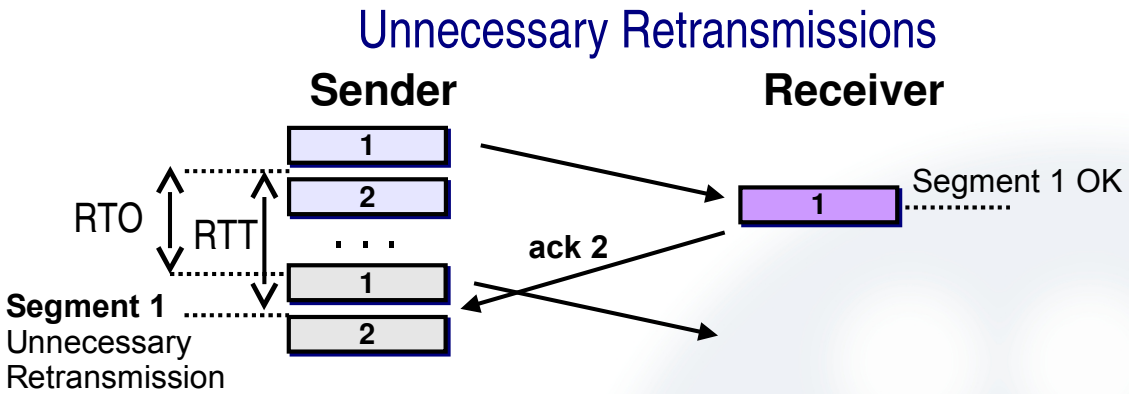



- We can have a disordered reception of segments.
- Note that in this case duplicate ACKs are sent.
- What to do with disordered segments? Store or discard them?

- Discard segments:
  - The recevier is simple.
- Accept the received segments within a valid window.
  - The receiver is a little more complex but many retransmissions can be avoided.
  - Used by TCP.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Retransmission TimeOut (RTO)




- The value of the retransmission timeout (RTO) is critical.
- A wrong *RTO* (*RTO* $<$ *RTT*) can cause unnecessary retransmissions.
- The problem is that the RTT in a real TCP/IP network is variable.

# Legacy RTO Estimation

- RTT Estimation (legacy algorithm):
  - $t_s$ is the departure time of each sent segment.
  - $t_a$ is the arrival time of the first ACK that acknowledges a segment.
  - The previous times are used to calculate a measure of the rtt for each segment:

$$rtt \leftarrow t_a - t_s$$

  - Then, measures are averaged using an "exponentially weighted moving average":

$$srtt \leftarrow \alpha \cdot srtt + (1 - \alpha) \cdot rtt$$

    Where $\alpha \approx 0.8$.
- $RTO = f(srtt)$:
  - The RTO is updated for each received ACK:

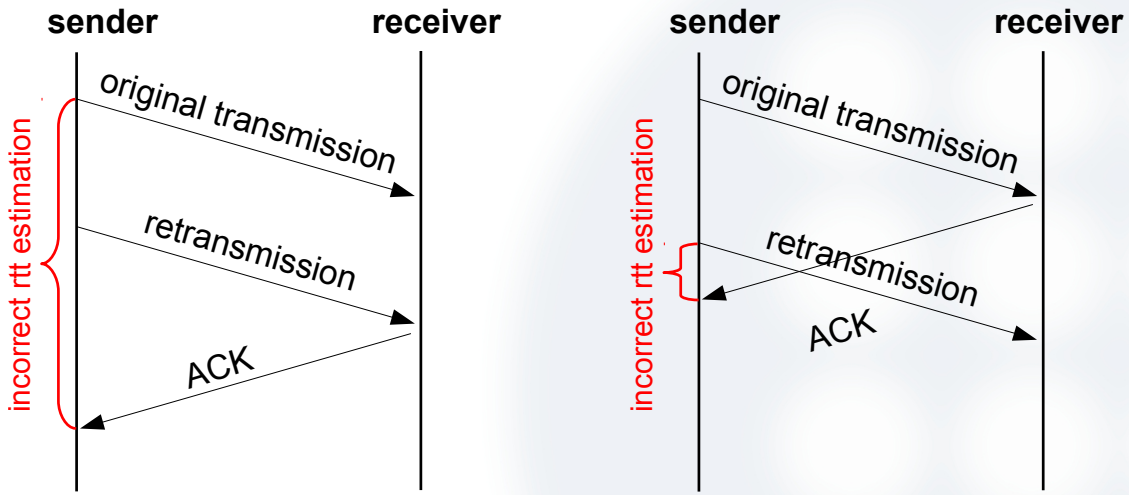
$$RTO \leftarrow \beta \cdot srtt$$

    Where $\beta = 2$.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Karn Algorithm

- There is a problem of acknowledgement ambiguity.
- When there is a retransmission, it is impossible to know whether the received ACK comes from the original or from the retransmitted segment.
- If we use the wrong assumption, the rtt measure will be incorrect:




- **Karn algorithm**:
  - Do not measure the rtt for retransmitted segments.
  - For every retransmission, the RTO is doubled (exponential backoff).
  - Typical values: $RTO_{max} = 64\,\mathrm{s}$ and $\#RTX_{max} = 12$.

# Jacobson RTO Estimation

- From Jacobson (1988).
- Introduces the variance for computing the rtt:

$$rttvar = (1 - \delta) \cdot rttvar + \delta \cdot |srtt - rtt|$$

$$srrt = (1 - \alpha) \cdot srtt + \alpha \cdot rtt$$

$$RTO = srtt + k \cdot rttvar$$

- Empirically the following values were selected:

$$\alpha = 1/8 \qquad \delta = 1/4 \qquad k = 4$$

- We can use Karn or the TCP “time-stamp” option to avoid the problem of the ACK ambiguity.
- In practice, many TCP implementations use RTOs multiple of 200, 500 or 1000 ms.
- In addition, when there is not any measure available, implementations try to be ”conservative“ by utilizing RTOs in the order of seconds.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

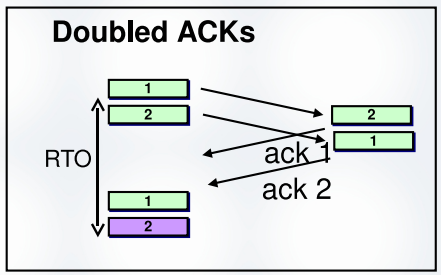
# Fast Retransmit I

- When a segment is lost, the corresponding ACK will never arribe to the sender.
- Then, in the legacy TCP (RFC793), the sender waits for the expiration of the RTO and retransmits the segment and subsequent segments using the Go-back-N mechanism.
- However, probably, many (or all) subsequently transmitted segments might have correctly arrived at the receiver.
- For each received segment, the receiver sends a dupACK to notify that it is waiting a missing fragment.
- TCP Tahoe decode dupACK as an indication that the segment was lost.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Fast Retransmit II

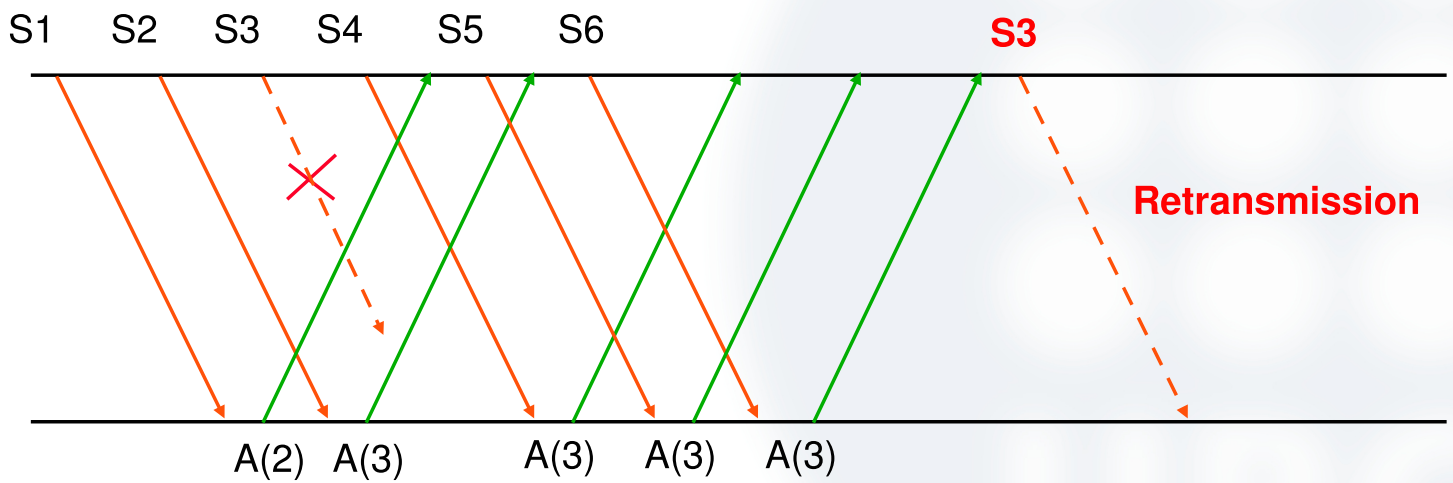
- Fast retransmit. 1997 Stevens Algorithm
- Idea: Detection of losses before RTO expires, so start rtx as soon as possible!



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Fast Retransmit III

- RTO is usually estimated and can be really large (even several times the RTT on congested networks).
- When 4 ACKs are received with the same sequence number (1 ACK + 3 dupACKs) implies the loss of the segment that follows the sequence number.



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# ACK Generation I

- Problem:
  - On an interactive program, such as telnet, it can occur that a segment might be sent for each typed character.
  - This would generate a high overhead (4 segments por character as telnet echoes the received data).
- Solution:
  - To wait 200 ms before the recognition.
  - In this way it is possible to take advantage of the echoed data with the ACK (only three segments exchange).
  - The duplicate ACKs are not delayed.
- Most of the implementations send an ACK every two correctly received segments (without error & ordered).
- Usually, in a non-normal context (disorder, after an error, etc.) an ACK is immediately sent for each segment correctly received.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# ACK Generation II

| Event | TCP Receiver action |
|---|---|
| Ordered segment arrival, no holes. Delayed ACK. Wait for 500ms (and 200ms usually and the rest of data already recognized). For the next segment. If no segment, send an ACK. | |
| Ordered segment arrival, no holes,<br>. One delayed ACK unresolved | Immediately send a cumulative ACK |
| Disordered segments, where<br><br>Seq Num > expected. Hole detection. | Immediately send a duplicate ACK<br>indicating the expected SN |
| Segment arrival that fills the hole. | Immediately send an ACK whenever the<br>segment really fills the hole |

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Outline

1 Net Apps Intro

2 UDP

3 TCP
- Connection Management
- Flow Control
- Error Control
- TCP segment



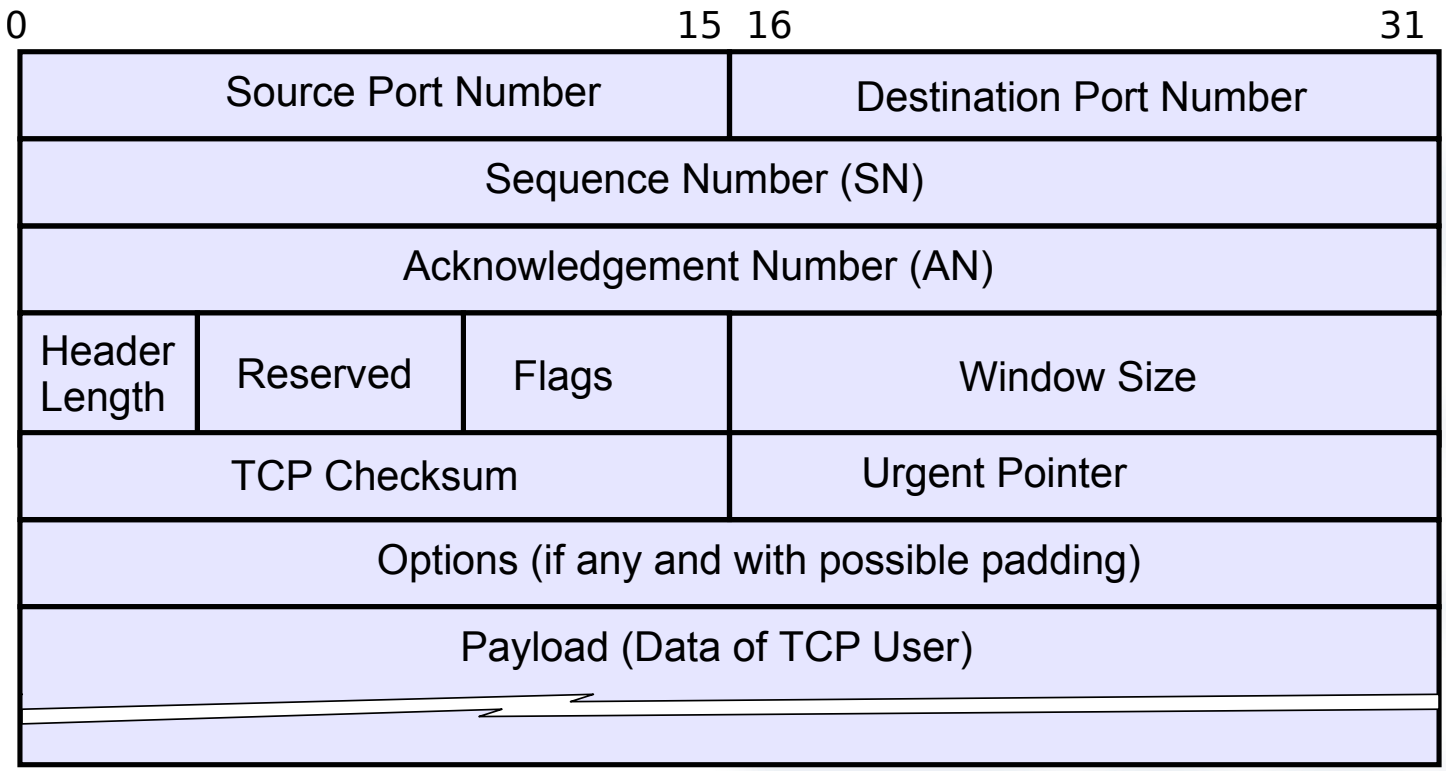
Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# TCP Segment I

- The TCP segment is the PDU (Protocol Data Unit) of TCP.
- The header of the segment is used in TCP for:
  - Identifying source and destination processes.
  - Establishing, releasing and reseting connections.
  - For transferring data in the segment's payload including error and flow control.
  - For the flow control the advertised window can be modified.
  - Etc.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# TCP Segment II



Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

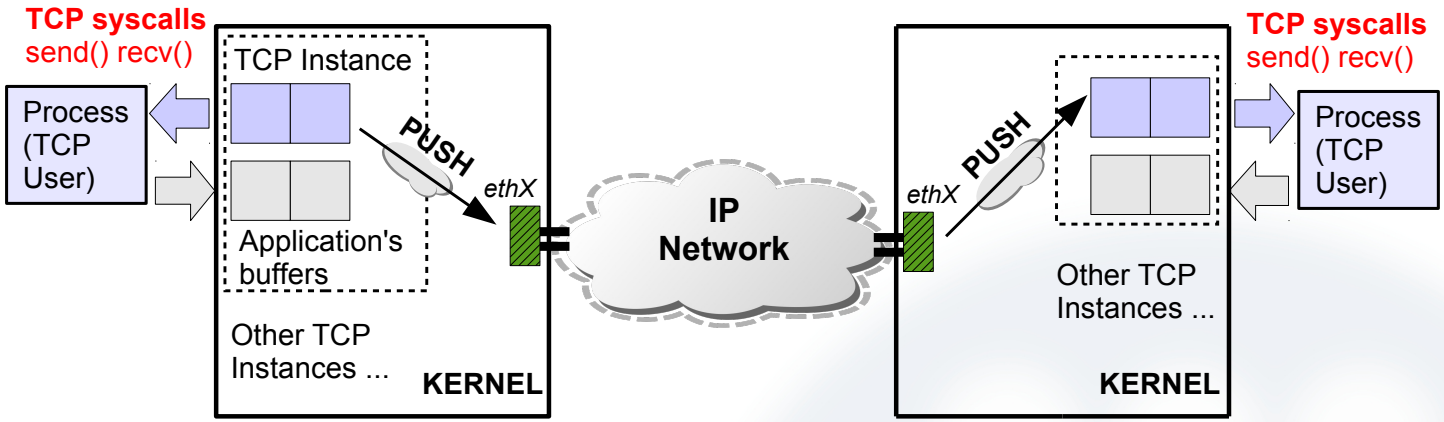
# TCP Fields (Not Yet Commented)

- **Header Length.**
  - This is a 4 bit field.
  - Shows the number of 32 bit words (4 bytes) in the header.
  - Also known as the Data Offset field.
  - The minimum size header is 5 words (20 bytes).
- **Reserved.**
  - This is a 6 bit field.
  - Not used at the present and it is always set to 0.
- **Control Bit Flags.** The flags are used to govern the process of connection establishment, data transmissions and connection termination.
  - **URG.** Activate urgent pointer.
  - **ACK.** Activate ACK.
  - **PSH.** Activate push function.
  - **RST.** Reset the connection.
  - **SYN.** Synchronize SN.
  - **FIN.** No more data from the sender.

# PUSH Flag I

- TCP transfers a stream of bytes, not of messages.
- Applications use the `send()` syscall to write bytes to the output application's buffer that is read by the TCP instance at the kernel.
- The fact that an application sends two 1000 byte messages does not mean that the sending TCP instance will send two 1000 bytes TCP segments.
- Sometimes users need to be sure that all the data they have submitted to the TCP has been transmitted.
- A push causes that the data is made available to the receiver by writing it into the application's buffer.

# PUSH Flag II




- The sending process should indicate in `send()` whether to set or not the PUSH flag.
- In practice, the main implementations of TCP sockets (e.g. UNIX BSD sockets) do not implement this.
- They set automatically this flag.
- For example, let's consider that we have established a TCP connection over Ethernet and that we want to send 2000 bytes.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# PUSH Flag III

- If the sender uses `send()` to send a payload of 2000 bytes like:

```
send(socket, buffer, 2000, ...);
```

- This results in 2 TCP segments (IP datagrams).
- The first carrying a payload of 1460 bytes (considering Ethernet MTU=1500 bytes).
- The second carrying a payload of the remaining 560 bytes, with a PSH bit set.
- In contrast, having the caller invoke `send()` twice to send the same data:

```
send(socket, buffer, 1000, ...);
send(socket,buffer+1000, 1000, ...);
```

- Results in 2 TCP segments, both having 1000 bytes of payload, and both having the PSH bit set.

# Urgent Mode I

- Out-Of-Band (OOB) data is data delivered with higher priority than ordinary data.
- OOB data means an exceptional condition.
- Use `MSG_OOB` in the `send()` syscall:

```
send(s,buf,len,MSG_OOB);
```

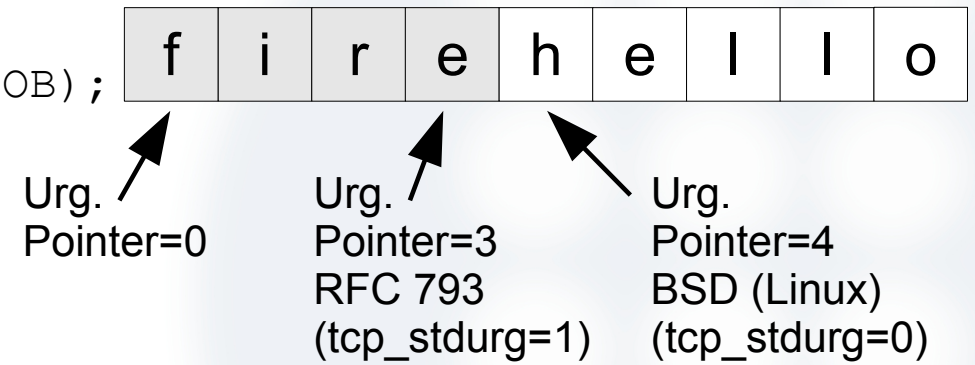
- To read urgent data use `recv()` syscall also with the `MSG_OOB` flag.
- When the kernel finds a socket with segments containing OOB data, it sends a SIGURG signal to the process that owns the socket.
- If the URG flag is set, the receiving station evaluates the urgent pointer, a 16-bit field in the TCP header.
- There are two interpretations of the urgent pointer.

# Urgent Mode II

- The standard (RFC 793) says that the urgent pointer points the last byte of out-of-band data.
- The BSD implementation says that the urgent pointer points after the last byte of out-of-band data.

- In Linux, the file /proc/sys/net/ipv4/tcp_stdurg defines the behavior (0=BSD,1=RFC).

```
send(s,hello);
send(s,fire,MSG_OOB);
```




- Urgent data is not employed much by modern protocols.
- Telnet perhaps is the most representative example (for echoing characters).

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# MSS Option

- MSS (Maximum Segment Size)
- The MSS option is used to announce the maximum number of bytes that we accept in a single TCP segment (with or without fragmentation).
- This option is typically sent in the first SYN segment.
- The Maximum Segment Size (MSS) is an announcement not a negotiation.
- It is sent from the data receiver to the data sender saying:
  ”I can accept TCP segments up to size X“.
- Each direction of data flow can use a different MSS.
- Typically, the MSS option is established by the OS Kernel using the output interface MTU.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Time-stamp Option

- Allows the transmitter to include a time reference in a TCP segment.
- The receiver echoes the time-stamp.
- With this option, the sender can estimate the segment round trip time (RTT).
- This option is utilized for a precise estimation of the RTT.
- Remember that the RTT is used to estimate the RTO, which is a critical parameter of the error control of TCP.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Window Scale Option I

- This option is defined in RFC 1323 which deals with long fat networks.
- The advertised window counts bytes and it has a size of 16 bits.
- With 16 bits, we have a maximum BDP of: $2^{16}\,\text{bytes} = 2^{6}\,\text{Kbytes} = 64\,\text{Kbytes} = 65535\,\text{bytes}$.
- The TCP window scale option is needed for efficient transfer of data when the bandwidth-delay product (BDP) is greater than 64 Kbytes.
- By using the window scale option, the receive window size may be increased up to a maximum value of $2^{14} * 2^{16} = 1,073,725,440\,\text{bytes}$.
- This is done by specifying a one byte shift count in the header options field.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# Window Scale Option II

- A maximum value of 14 may be used for the shift count value.
- Example:
  - If a 2 Mbps transmission line was used over a satellite link with $RTT = 513\,\text{ms}$.
  - Then, $BDP = 2000000 \times 0.513 = 1026000\,\text{bits} = 128250\,\text{bytes} = 125.24\,\text{Kbytes}$.
  - Using a maximum buffer size of 64 Kbytes only allows the buffer to be filled to (65535 / 128250) = 51% of the theoretical maximum speed of 2 Mbps.
- Note that with the window scale we can would allow a single TCP connection to transfer data over the example satellite link at 2 Mbps utilizing all of the available bandwidth.

Net Apps Intro
Transport Layer Motivation
Client/Server Model
Socket API
netstat
netcat
UDP
UDP Datagram
UDP Applications
TCP
Connection Management
Flow Control
Error Control
TCP segment

# SACK Option

- TCP allows the user to utilize a selective acknowledgement mechanism as error control.
- The SACK option is required to implement this mechanism.
- A SACK (Selective ACK) expresses a positive recognition of a block of bytes.
- SACK error control accelerates the error recovery.
- In TCP options up to three SACKs can be carried.